

# NYCU-ECE DCS-2026

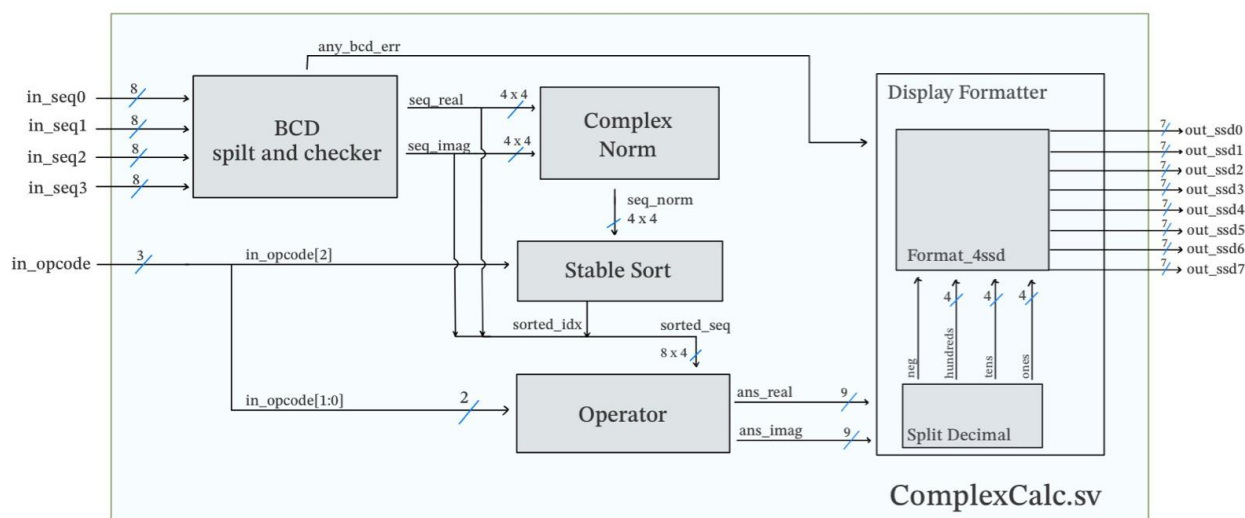
## HW01 Report

Name: 楊庭瑞

Student ID: 113511266

Server Account: dcs151

### Architecture



### Design Methodology

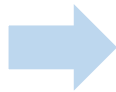
#### 設計架構:

1. 將 4 個 Input 輸入 BCD\_split\_and\_checker 模組，並輸出實部與虛部，有出現任何錯誤，則將 any\_bcd\_err 輸出為 1 (超過 9 或實虛部皆 0)。
2. Calculate Norm 計算出四個複數的絕對值，並將其傳入 Stable Sort 與 in\_opcode[2] 進行排序
3. 排序後的數列經過 Operator，由 in\_opcode[1:0] 決定將 ANS 進行何運算。
4. 將得到的 ans\_real, ans\_imag，傳入 Split Decimal 轉換成 BCD 的百位十位個位，及一個是否有負號的 neg 進行判斷，最後經過 Format\_4ssd 決定輸出到七段顯示器為何。

#### 面積優化:

1. 將每個變數的 bit 最小化，以免多餘的運算占用空間
2. Calculate Norm 原本使用助教所給牛頓法，但由於數值皆在 0~9，因此直接建平方表，及開根號表，省下很多乘法器，除法器，改變 StableSort 的演算法，利用 idx 減少計算量。
3. 將 ans\_real, ans\_imag 轉成十進位的個十百避免使用除法，找餘數，而是利用區間判斷來找出個十百位數。
4. 使用 Case，預設值寫 X(ex: default: 4'dx)，有時候能縮小面積，但有時候會變大。

```
// Step5: OPcode stable sort
StableSort norm_sorter(
    .in_seq0(in_seq0),
    .in_seq1(in_seq1),
    .in_seq2(in_seq2),
    .in_seq3(in_seq3),
    .in_norm0(seq0_norm),
    .in_norm1(seq1_norm),
    .in_norm2(seq2_norm),
    .in_norm3(seq3_norm),
    .in_opcode(in_opcode[2]),
    .out_seq0(sorted_seq[0]),
    .out_seq1(sorted_seq[1]),
    .out_seq2(sorted_seq[2]),
    .out_seq3(sorted_seq[3]),
    .out_norm0(sorted_norm[0]),
    .out_norm1(sorted_norm[1]),
    .out_norm2(sorted_norm[2]),
    .out_norm3(sorted_norm[3])
);
```



```
// Step5: OPcode stable sort
StableSort norm_sorter(
    .in_norm0(seq_norm[0]),
    .in_norm1(seq_norm[1]),
    .in_norm2(seq_norm[2]),
    .in_norm3(seq_norm[3]),
    .in_opcode(in_opcode[2]),
    .out_idx0(sorted_idx[0]),
    .out_idx1(sorted_idx[1]),
    .out_idx2(sorted_idx[2]),
    .out_idx3(sorted_idx[3])
);

assign sorted_seq[0] = {seq_real[sorted_idx[0]], seq_imag[sorted_idx[0]]};
assign sorted_seq[1] = {seq_real[sorted_idx[1]], seq_imag[sorted_idx[1]]};
assign sorted_seq[2] = {seq_real[sorted_idx[2]], seq_imag[sorted_idx[2]]};
assign sorted_seq[3] = {seq_real[sorted_idx[3]], seq_imag[sorted_idx[3]]};
assign sorted_norm0 = seq_norm[sorted_idx[0]];
assign sorted_norm2 = seq_norm[sorted_idx[2]];
```

原本 StableSort 將整個 seq, norm 都放進去排列，後來利用 2 bit idx 傳出對應的指數減少面積

## Reflection

對於還不太熟悉 Verilog 的我來說，在撰寫這次作業的過程中獲得了滿滿的挫折感。很難想像自己才剛學會一些基本的 Verilog 語法，就需要完成這麼大型且複雜的題目。整個題目包含許多步驟與模組，需要先理解題目的邏輯流程，才能開始思考如何實作；即使腦中已經有了一些想法，實際要用 Verilog 寫出來時仍然常常不知道該如何下手。

在整個實作過程中，最困難的部分可能是 Debug。當結果不正確時，很難判斷到底是哪一個步驟出了問題，也不太確定自己前面的實作是否正確。雖然 nwave 的 Debug 看起來很強大，但我目前對它還不太熟悉，因此在檢查訊號時花了不少時間。對於像這種需要分成許多步驟完成的大型題目而言，整個撰寫過程其實相當折磨，主要還是因為難度真的很高。

不過幸好在目前的階段，大部分遇到的問題仍然可以透過 AI 的協助找到解決方法，也因此才能順利完成作業。希望透過這次的練習，自己能夠逐漸熟悉 Verilog 的設計方式，也希望助教能夠手下留情。

## AI-use Log

- What tools/models you used? (e.g., Gemini, ChatGPT, Copilot, ...)  
Copilot Agent + GPT-5.3-Codex
- Key Prompts: 3–5 primary prompts (paste the original text or conversation link)
  - (貼程式碼)幫我檢查是否有語法錯誤
  - (貼程式碼)這一段是否可以優化面積或結構，或是改成...去實作
  - (貼程式碼)出現了.....的問題應該怎麼辦才好。
  - [https://docs.google.com/document/d/1P7BJ\\_KfGKmxrKtVfCvii05r4QE70P0kYxr-hPXwIxbY/edit?usp=sharing](https://docs.google.com/document/d/1P7BJ_KfGKmxrKtVfCvii05r4QE70P0kYxr-hPXwIxbY/edit?usp=sharing)
- What and why you adopt or reject? How you verify generated output?  
確保 AI 生出來的 code 自己看得懂，也認為比現在版本的還要好，盡可能避免在看不懂的時候直接採用。-